Porting institutional deep-learning seismic phase pickers to PyTorch: a practical guide to TensorFlow–PyTorch weight transfer

C. Skevofilax, A. Savvaidis
The University of Texas at Austin

May 4, 2026

Abstract

Seismic research groups train their own phase-picking models using whichever machine learning framework that fits their process workflow, and TensorFlow/Keras remains a popular choice. At the same time, the modern open-source seismology ecosystem has largely standardized around PyTorch, with SeisBench reinforcing this shift by adopting PyTorch as its exclusive framework. This exclusivity creates a practical challenge for researchers who have existing models using other frameworks: how can these models be transferred to PyTorch without retraining? In practice, transferring model weights can be difficult, and without a reliable conversion process, research groups often struggle to integrate their prior work into modern toolkits.

In this work, we present a practical methodology for converting transformer-based phase pickers from TensorFlow to PyTorch, using EQCCT as a case study. We address the key challenges required for reliable conversion: weight-loading inconsistencies in Keras, ambiguity when mapping dense layers with square matrices, incomplete output branches, and device-dependent numerical differences. We validated the converted model using both layer-by-layer comparisons and end-to-end evaluations on 100,000 real waveforms from the TXED and STEAD datasets made accessible by SeisBench's API. The mean absolute error between the TensorFlow and PyTorch probability traces is on the order of 10^{-9} when processing on only CPUs, and 10^{-7} to 10^{-6} on GPUs, with worst-case discrepancies below 10^{-2}. These results demonstrate that our methodology faithfully ports the original EQCCT model, providing seismic researchers with a reliable roadmap for framework migration and fostering better collaboration within the seismic community.

1 Introduction

Deep learning-based seismic phase picking models have rapidly transformed seismology over the past decade. Models like PhaseNet ([43]), EQTransformer ([18]), and EQCCT ([29]) have improved seismic event detection and catalog automation to the point where they are now being integrated into real-time production workflows ([20]; [34]). By standardizing these models into a Python library, SeisBench (SB, [38]) has lowered the barrier between research and application, fostering a community where researchers can more easily share their work with a broader audience as well as quickly utilize state-of-the-art technologies.

Like many research institutions and open-source projects, SB has adopted PyTorch (PT, [24]) as its standard machine learning framework. This adoption reflects a broader trend in the research community, where PT now accounts for approximately 85% of deep learning papers, according to

recent surveys ([32]). By standardizing on a single framework, SB avoids the complex code handling and maintenance of multiple frameworks, ensuring a more predictable and unified workflow.

However, seismic networks and research groups like the Texas Seismological Network (TexNet, [31]) have already trained models using alternative frameworks such as TensorFlow (TF, [1]). Because of SB's PT-based approach, these groups cannot integrate their models into the toolkit until they convert them to a PT format, which limits the potential for advancing seismic research and real-world applications by limiting its potentially larger audience.

While tools like ONNX ([22]) exist for the weight transfer process, practical guides remain rather scarce and domain-specific issues can be hard to overcome. ONNX can fail with complex model designs, leaving researchers with little literature to fall back on. Empirical studies show that conversion can trigger accuracy degradation, performance swings, and numerical inconsistencies ([14]; [42]), further highlighting the difficulty of this challenge. Having had to navigate these hurdles ourselves when porting EQCCT to PT, we present this work as a practical guide for other seismic researchers facing similar issues to help streamline the weight transfer process.

We chose EQCCT as our case study because of its production-level status and complex dual-branch architecture. TexNet currently uses EQCCT in its production operations, and other seismic networks have expressed interest in using it, however without a SB-based accessor, usage remains limited. Additionally, its separate P and S models use distinct convolutional structures, which further test our conversion process.

Our contribution is primarily methodological. We demonstrate how to map Keras HDF5 checkpoints to PyTorch state dictionaries, establish numerical parity through layer-wise validation, and verify model fidelity using real waveform data. We also address practical nuances like device-specific numerical drift and weight-loading ambiguities that frequently arise during framework migration.

2 Background

2.1 EQCCT architecture

EQCCT ([29]) is a transformer-based phase picker built for production-level earthquake monitoring. The architectural features a convolutional front end, patch tokenization, transformer encoder blocks, and a 1D convolutional head that uses sigmoid activation to generate probability traces for P and S phase arrivals. Because of the different characteristics that make up P and S waves, EQCCT opted to use separate models with different depths and layer structures for phase extraction. This dual-branch design makes it a rigorous test case for framework conversion.

For a deeper look at the specific training procedures, architectural choices, and accuracy metrics, refer to [29].

2.2 SeisBench, production deployment, and numerical parity

SeisBench [38] provides researchers with a modular framework for accessing standardized seismic datasets and PyTorch-based machine learning models via a Python library. By allowing users to contribute their own models, the toolkit standardizes how solutions are shared and benchmarked across the field, effectively streamlining how machine learning is applied to seismology. Because of these benefits, it is becoming essential for models like EQCCT to integrate models into the SeisBench ecosystem as native PyTorch modules.

That said, many models not yet integrated into SeisBench or trained using PT are already running in production applications. EQCCT, for example, serves as the primary detection algorithm for

near-real-time ([6]) and real-time detection systems ([20]) via TF-based accessors. While this works from a technical standpoint, this sidesteps the standardization SeisBench provides and increases the technical burden for users unfamiliar with managing multiple software environments. These workarounds ultimately limit the broader adoption of these state-of-the-art models and underscore the value of making them available through SeisBench.

2.3 Challenges in framework conversion

Model weight transfer is a notoriously difficult task, even for experienced programmers. Failures typically stem not from misunderstanding the original architecture, but rather from subtle, framework-specific implementation details that can be hard to detect. These complications typically arise from several key areas:

First, architectural mismatches can occur when a model is rewritten manually. Although this issue may seem trivial, it is complicated by differences in how frameworks define and execute computational graphs, which can obscure internal logic that affects numerical behavior. For example, TF's static-graph approach can fuse operations or reorder them for optimization purposes, making it difficult to reproduce the exact same sequence of computations within PT's dynamic execution environment. Additionally, changes in framework versions, such as deprecated functions and or modified operator semantics, require users to match the implementation to the exact version used during training [42], adding another level of complexity.

Second, frameworks often differ in their mathematical definitions of operations, which directly affects numerical parity when transferring weights. For example, MaxPool2d in PT and PaddlePaddle, another machine learning framework, appear quite similar, but PT's MaxPool2d supports dilation while PaddlePaddle does not. This is critical, as dilation effectively expands the pooling window by increasing the spacing between kernel elements, enlarging the receptive field, altering both the output size and the features extracted ([42]). Similarly, TF and PT use different values for key parameters such as epsilon in batch normalization or numerical stability constants in layer normalization ([37]). Although these inconsistencies may seem minor, the errors they introduce can compound across layers and lead to large probability discrepancies between model solutions. They can be difficult to identify as most of their definitions are defined in C/C++ kernels rather than being exposed on the Python API level ([37]).

Third, frameworks tend to use unique tensor layout definitions. TF uses a channel-last approach (NHWC format), while PT uses a channel-first approach (NCHW format). This requires users to transpose weight matrices during conversion. For dense layers, Keras stores kernels with shape (in, out), while PyTorch Linear layers expect (out, in). This is critical, as even when weight matrices are square, the transpose must be applied to maintain mathematical correctness. Failing to do so affects numerical parity and can silently degrade model performance, as mentioned earlier.

Finally, GPU execution introduces genuine numerical divergence due to kernel selection, compiler-driver fusion (such as XLA in TensorFlow), and mixed-precision arithmetic. As a result, using the same weights does not guarantee bitwise-identical outputs across devices, as will be discussed later.

3 Methods

3.1 Conversion workflow overview

The conversion workflow proceeds in three stages. First, we manually write a canonical implementation of each model architecture in PT to ensure every layer is accurately defined and to provide a clear target for mapping the original TF weights. Second, we extract the weights from the Keras

HDF5 checkpoints and transform them to match the expected PT tensor layouts. Finally, we validate the conversion by comparing the PT model against the TF original, starting with controlled tests on random inputs and progressing to real seismic waveforms from SeisBench datasets. The subsections below follow this progression and highlight the main challenges encountered at each stage.

3.2 Architectural alignment between frameworks

Before extracting any weights, we recommend first faithfully porting the full architecture from the original TF definitions to PT. This ensures that when weight tensors are transferred, they will land in the correct mathematical operations, with the right depthwise ordering, tensor layouts, and activation functions. In our implementation, the P- and S-branch EQCCT models share a single Python module that serves as the canonical PT implementation of the original Keras graph. Each branch's `state_dict`, a Python dictionary that maps layers to their parameter tensors, is aligned to the naming conventions used in Keras so that every layer has a clear, one-to-one counterpart. For example, multi-head attention output projections are stored under `attn.o`, MLP layers under `mlp.fc1` and `mlp.fc2`, and the extra S-branch convolution stacks, referred to as `extra_pre` and `extra_post` in the TF graph, are copied into their corresponding PT entries.

Naming accuracy alone is not enough, however. Small implementation details embedded in the Keras computational graph also need to be replicated, and these can be easy to overlook. One example is the `transformer_units` tuple, which in Keras applies a GELU activation after each of the two dense layers in the MLP block. The PT `TransformerMLP` must follow that same sequence precisely, so each linear layer is followed by GELU and dropout in the same order as the original Keras loop. To ensure that output comparisons are meaningful, we call TF with `training=False` and run the PT model in evaluation mode so that dropout and batch normalization behave consistently across both frameworks during inference.

3.3 Weight extraction and transformation

Keras stores model weights in HDF5 files organized as a nested hierarchy. To make these weights usable in PT, we flatten each file into a dictionary whose keys resemble filesystem paths, such as `patch_encoder/dense/kernel:0` and `multi_head_attention/query/kernel:0`. We then apply a normalization step that collapses any duplicated path segments that can appear in Keras exports, converting a key `conv1d/conv1d/kernel:0` to `conv1d/kernel:0`, which keeps tensor ordering consistent for both manual inspection and the automated pairing logic.

Table 1 summarizes the main structural differences between Keras and PT tensor layouts, and these transformations need to be applied carefully and consistently. Without close attention to layout, middle layers can end up with incorrect weights even when early layers appear to pass spot checks.

A persistent issue that occurred involved the dense layers in the P-branch MLP blocks. Keras stores dense kernels with shape (in, out), whereas PT Linear layers expect (out, in), meaning a transpose must be applied in every case, including when both the original matrix and its transpose share the same shape. For square matrices like the 40x40 blocks in the MLP, a naive implementation that simply accepts the first matching shape without transposing will silently produce incorrect weights while other unrelated layers appear fine. Enforcing the transpose rule unconditionally for all dense layers resolved the persistent activation mismatch we saw in the first transformer block.

A second challenge was specific to the S branch. The routine responsible for loading the S-branch HDF5 checkpoint into the PT module had to explicitly copy the `picker_S` Conv1D kernel and bias

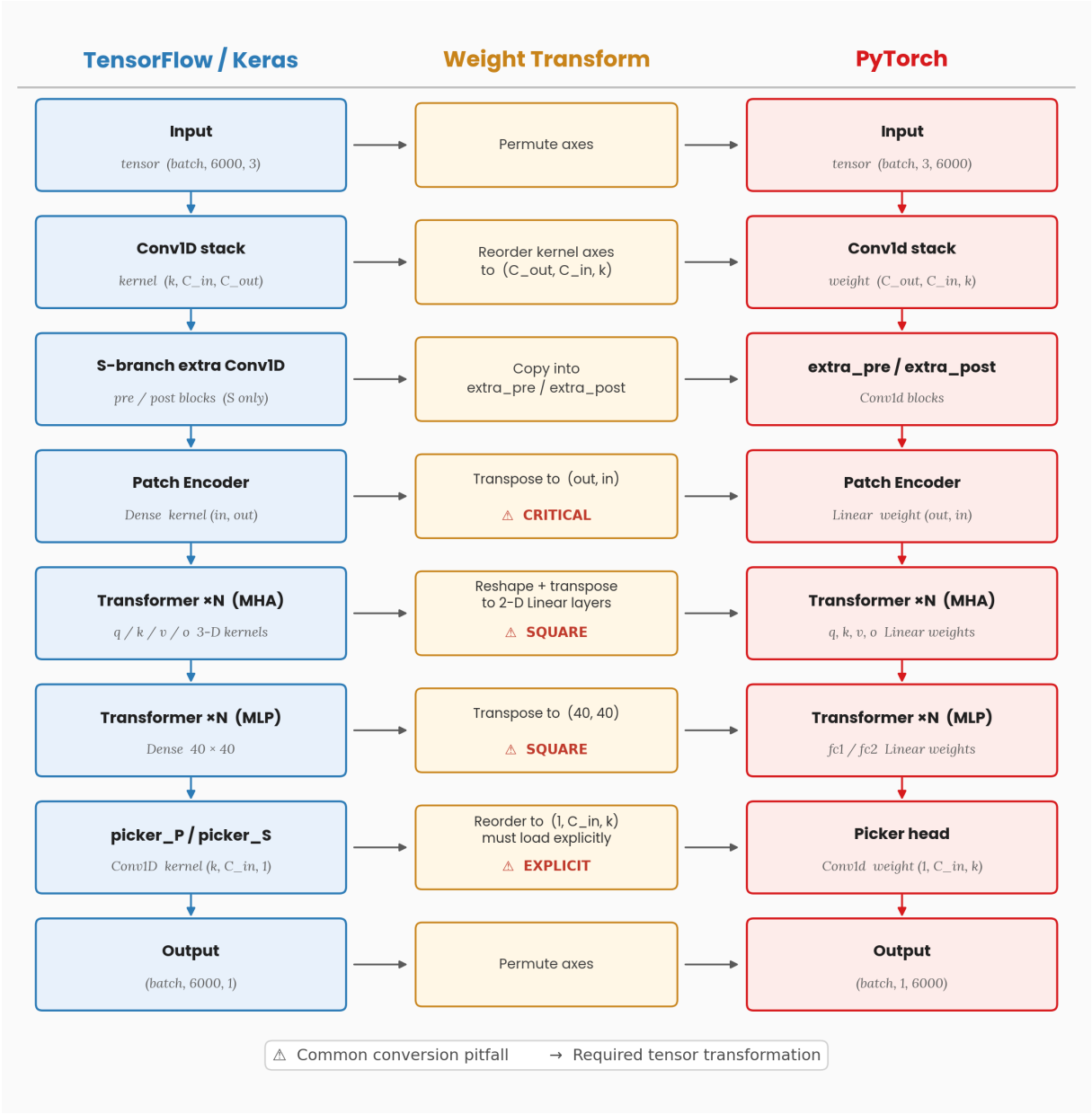

Figure 1: TensorFlow-to-PyTorch architectural correspondence for EQCCT. Three parallel columns show the Keras-style module stack (left), the explicit tensor and weight remapping applied during transfer (center), and the PyTorch modules (right), row by row from the 6000-sample ZNE input through the Conv1d front end (with S-only extra stems where applicable), patch encoder, repeated transformer multi-head attention and MLP blocks, and the convolutional picker to output traces. The middle column flags the steps that most often break naive porting, notably dense kernel transposes for patch and MLP layers, reshaping and transposing non-square/square multi-head projection kernels into paired `Linear` weights, and Conv1d kernel axis reordering.

Table 1: Keras versus PyTorch tensor layout for EQCCT weight transfer.

Keras concept	PyTorch module	Typical transform
Dense kernel, shape (in, out)	`Linear.weight`, shape (out, in)	Transpose
Conv1D kernel $(k, c_{\text{in}}, c_{\text{out}})$	`Conv1d.weight` $(c_{\text{out}}, c_{\text{in}}, k)$	Permute axes
BatchNorm moving statistics	`BatchNorm1d` buffers	Copy after key alignment
MultiHeadAttention Q, K, V, output	Separate `Linear` projections	Reshape 3D Keras kernels to 2D per projection rules

into the model's final output head (`head.conv`). If that step was skipped, the backbone would load correctly while the output layer remained at random initialization, a failure that was easy to miss until end-to-end validation caught it.

Weight loading in the reference TF model also required care. Calling TF's `load_weights()` without name-based assignment binds arrays to layers in graph construction order. If that order does not match how variables are stored in the checkpoint, later layers, particularly batch normalization and attention, can silently receive the wrong tensors while early convolutions look correct. To avoid this, we prefer strict name-based loading whenever the checkpoint supports it, using a short fallback chain when it does not.

Our first attempt uses `load_weights(..., by_name=True, skip_mismatch=False)`, which requires every variable to match a checkpoint entry by both name and shape. If any mismatch is found, loading stops immediately rather than silently assigning weights to wrong layers. If that fails, we try `by_name=True, skip_mismatch=True`, which still pairs by name but skips any variable with no compatible checkpoint entry, including those affected by Keras 3 versus legacy layout differences in multi-head attention. Variables that are skipped remain at their initialization values, and since reproducing those exactly depends on framework defaults, random seeds, and undocumented initialization logic, we treat any skipped variable as a potential source of error. If neither named strategy succeeds, we fall back to positional loading via plain `load_weights(path)`, which binds arrays to layers in checkpoint order. This is the least reliable approach and can misassign deeper layers even when early convolutions look fine. Whichever strategy is used, we log it so that any deviation from strict loading is traceable.

To support reproducibility, Algorithm 1 describes the HDF5 flattening and path normalization step, and Algorithm 2 describes the dense-layer coercion rule that handles the square-matrix ambiguity described above.

Algorithm 1 (HDF5 flatten + path normalization). Walk the HDF5 file recursively and produce a flat dictionary mapping each normalized key to its NumPy array. For each dataset found at path p, read it as an array A, normalize p by collapsing any duplicated segments such as `conv1d/conv1d/kernel:0` to `conv1d/kernel:0`, and store the result as out[p] = A.

Algorithm 2 Map a Keras Dense kernel W to a PT Linear weight tensor with target shape tgt = (out, in). If W.T.shape matches tgt, use W.T, even when the matrix is square. If W.shape matches tgt instead, use W directly. Otherwise, raise a shape mismatch error and report the candidate shapes.

3.4 Validation strategy

To verify that the ported weights populated the PT graph correctly, we built a validation pipeline that allowed mismatches to be localized and fixed before moving onto large-scale waveform benchmarks.

The first stage consisted of random-input parity tests, where we passed identical NumPy tensors to both the TF reference and converted PT models and compared the resulting sigmoid probability traces using `numpy.allclose`-style checks. For individual parameter tensors such as kernels, biases, and normalization statistics, we used rtol = 1e-6 and atol = 1e-5, applied after the same layout transforms as the production loader. For intermediate activations at the conv stem, patch encoder, and transformer blocks, we relaxed these tolerances to 1e-4 for both relative and absolute error. The looser threshold reflects the fact that activations are inherently noisier than raw weights, as they accumulate small rounding differences across many combined operations including linear layers, softmax in attention, and nonlinearities, and they tend to span wider dynamic ranges than individual weight entries. These activation checks are therefore diagnostic tools for identifying where the graph first diverges rather than tests of bitwise parity.

Where useful, we also compared arrays from the Keras HDF5 checkpoint directly against loaded PT parameters after applying the same layout transforms as the production loader, focusing particularly on the convolutional stacks and the final Conv1D heads of both branches.

Building on this, we developed custom utilities, `tf_pt_p_trace` and `tf_pt_s_trace`, that systematically compare parameters and activations at each major architectural stage: after the convolutional front end, after the patch encoder, within each transformer block, and just before the picker head. An optional `block0-substeps` mode adds finer checkpoints inside the first transformer block, which we used to pinpoint exactly where outputs diverged when the MLP mapping was still incorrect.

Once the PT models passed these numerical checks, we proceeded to a dataset-scale benchmark using waveforms from the STEAD and TXED datasets accessed through SeisBench's API. We drew 50,000 six-thousand-sample windows from each dataset, all centered on waveforms containing both P- and S-wave arrivals within the window. Waveforms were resampled to 100 Hz and reorganized into ZNE component order. For each window, we ran the TF and PT versions of both the P and S models on the same input tensor, ensuring that every comparison was between matched model pairs on identical data.

For each window, we computed pointwise differences between the paired probability traces and tracked aggregate statistics using Welford's online method. These included MSE and MAE across all time samples and output channels, the mean and standard deviation of each window's maximum absolute error, the global worst-case absolute error across all windows, and the fraction of windows whose maximum error fell below fixed thresholds ranging from 1e-6 to 1e-2. All of these metrics were designed to quantify agreement between implementations rather than to measure pick accuracy.

We ran the full benchmark using both CPUs and GPUs. The CPU trials provided a stable baseline for verifying weight parity due to their deterministic execution. The GPU trials, on the other hand, introduced minor fluctuations via cuDNN and XLA optimizations. These were treated as device noise rather than evidence of mapping failures, as the CPU results remained consistent.

To make the staged validation actionable, Table 2 summarizes the validation gates we use, what they test, and what constitutes an informative failure.

4 Results

Table 3 summarizes the large-scale benchmark on the 100 000 SeisBench waveform samples discussed in Section 3.4. All windows completed without I/O or runtime errors.

On CPU, mean MAE between TF and PT stayed below 1e-8 for both branches, consistent with the mean MAE of 1.4e-9 reported in Table 3, and every window had a per-point maximum absolute difference below 1e-4 relative to the paired TF trace. This is the strongest evidence that the ported

Table 2: Validation gates and typical failure signals used in this study.

Gate	What it tests	Output	Typical failure signal
Random-input parity	End-to-end forward equivalence under controlled inputs	Max and mean |TF − PT| on outputs	Large mismatch indicates layout or missing weights
Weight trace	Correct parameter mapping (after coercion)	Per-layer allclose checks (rtol/atol)	Localizes wrong transpose/permute
Activation trace	Correct intermediate semantics (per stage)	Stage-wise max/mean |Δ|	Pinpoints first diverging block/op
SeisBench windows	End-to-end behavior on real waveforms	MSE/MAE, max |Δ|, fractions below thresholds	Surfaces device effects and rare worst cases

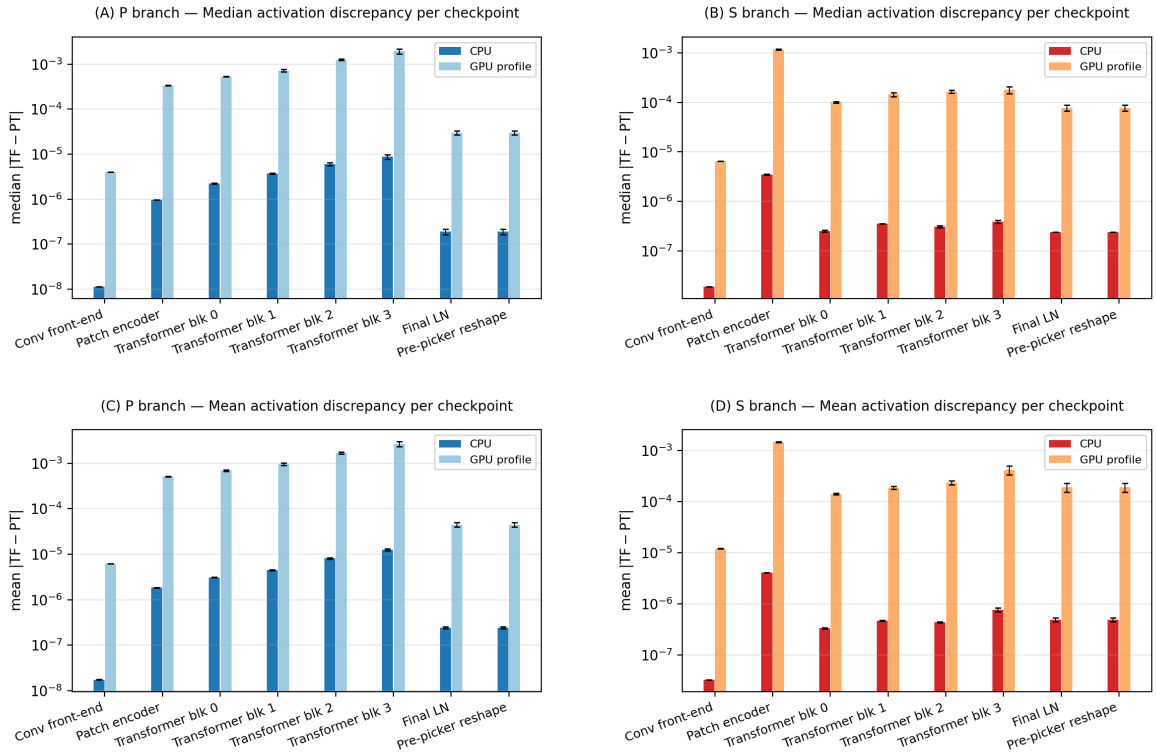

Figure 2: Layer-wise activation discrepancy between TensorFlow and PyTorch at checkpoints along EQCCT's forward pass. *Left column (A,C):* P branch; *right column (B,D):* S branch. *Top row:* geometric median statistic of |TF − PT| over activation elements at each stage, averaged across random Gaussian ZNE tensors (five seeds by default); error bars give the seed-to-seed standard deviation. *Bottom row:* mean |TF − PT| over elements with analogous across-seed error bars (y-axes on a log scale). Darker hues correspond to a CPU numerical profile and lighter hues to the GPU numerical profile.

PT graph faithfully reproduces the Keras reference when both frameworks use deterministic CPU kernels.

On GPU, discrepancies were larger but remained small in absolute terms for probability-valued outputs. Mean MAE stayed near 1e-7 for the P branch and 1e-6 for the S branch. 89.5% of P windows and 70.6% of S windows had a per-window worst-point difference below 1e-4, all P windows and 99.95% of S windows fell below 1e-3, and every window across both branches stayed below 1e-2. This gap relative to CPU is consistent with known variability introduced by cuDNN, XLA-style fusion, and GPU execution paths generally. Because the CPU run uses the same transferred weights and code paths as the GPU trials, this spread does not on its own indicate a separate weight-mapping error. With GPU discrepancies capped below 1e-2 and CPU errors below 1e-4, the CPU parity results remain the primary evidence that the mapping is correct.

Table 3: TensorFlow versus PyTorch numerical agreement on 100 000 SeisBench windows (TXED and STEAD, 50 000 per dataset, stride 1). MSE and MAE are averaged over windows; each window contributes one vector of errors over all time points in the P or S probability trace. *Mean max* $|\Delta|$ is the mean, over windows, of the maximum absolute difference within that window. *Global max* $|\Delta|$ is the worst case over all windows. Fractions are the share of windows with per-window maximum absolute difference below the stated tolerance.

Quantity	**Device**	**P branch**	**S branch**
Mean MSE	CPU	3.4×10^{-16}	2.2×10^{-16}
Mean MSE	GPU	1.1×10^{-11}	5.8×10^{-11}
Mean MAE	CPU	1.4×10^{-9}	1.4×10^{-9}
Mean MAE	GPU	2.3×10^{-7}	6.6×10^{-7}
Mean max $\lvert TF - PT \rvert$ per window	CPU	3.2×10^{-7}	2.4×10^{-7}
Mean max $\lvert TF - PT \rvert$ per window	GPU	4.7×10^{-5}	8.8×10^{-5}
Global max $\lvert TF - PT \rvert$	CPU	3.6×10^{-6}	3.9×10^{-6}
Global max $\lvert TF - PT \rvert$	GPU	8.8×10^{-4}	1.9×10^{-3}
Share of windows ($\max \lvert\Delta\rvert < 10^{-4}$)	CPU	100%	100%
Share of windows ($\max \lvert\Delta\rvert < 10^{-4}$)	GPU	89.5%	70.6%
Share of windows ($\max \lvert\Delta\rvert < 10^{-3}$)	CPU	100%	100%
Share of windows ($\max \lvert\Delta\rvert < 10^{-3}$)	GPU	100%	99.95%

5 Discussion

5.1 Implications for interoperability

As documentation for model weight transfer remains limited, by providing the seismic community with a practical, manual-style guide for moving models from TF to PT, we hope to make it easier for more research applications to find their way into unified tools like SeisBench, broadening user access to state-of-the-art phase picking technology.

The benefits of this kind of portability extend beyond toolkit integration. Real-time seismic event detection systems like SCMLPick ([20] depend on fast model inference to function in operational settings, and the performance differences between frameworks are directly relevant in that context. As shown in Table XYZ, the PT version of EQCCT is consistently faster than its TF counterpart on both CPU and GPU, uses less GPU memory, and on CPU avoids the startup overhead and device initialization complexity that can complicate TF in worker processes. These advantages suggest that migrating to PT carries meaningful practical benefits beyond ecosystem compatibility, particularly in real-time deployment scenarios.

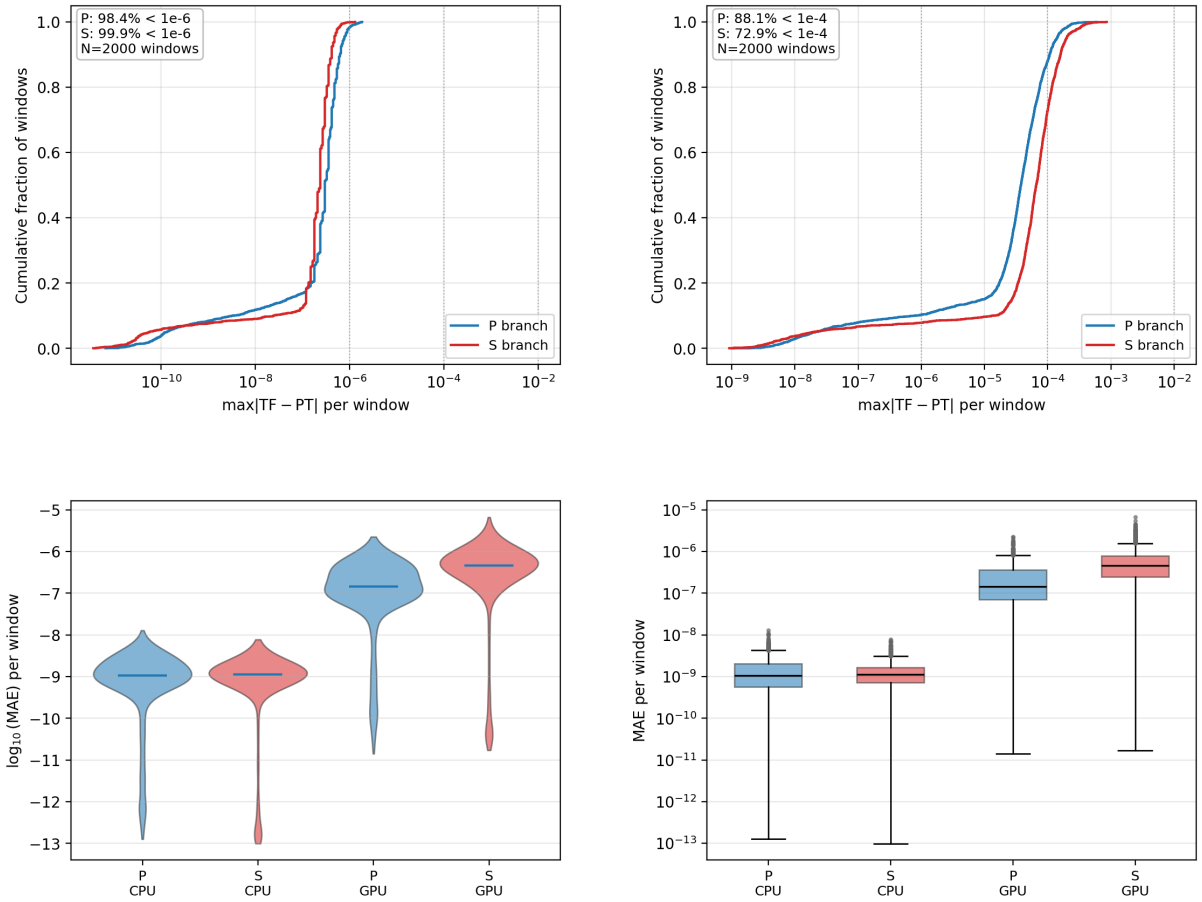

Figure 3: Per-window agreement between TF and PT on SeisBench windows. *Panels A and B:* empirical CDFs of max |TF − PT| on CPU (*A*) and on a GPU profile (*B*), with reference lines at 10^{-6}, 10^{-4}, and 10^{-2} (x-axis log scaled). Percentiles in-callout thresholds differ appropriately by device class. *Panel C:* violin densities of $\log_{10}$(MAE) where MAE averages |TF − PT| over samples within each window, for P/S branches on CPU versus GPU (y-axis logarithmic via $\log_{10}$). *Panel D:* box summaries of plain MAE on a log-scale y-axis across the four branch×device combinations. Color encodes branch (blue = P, red = S, repeated under CPU versus GPU panels).

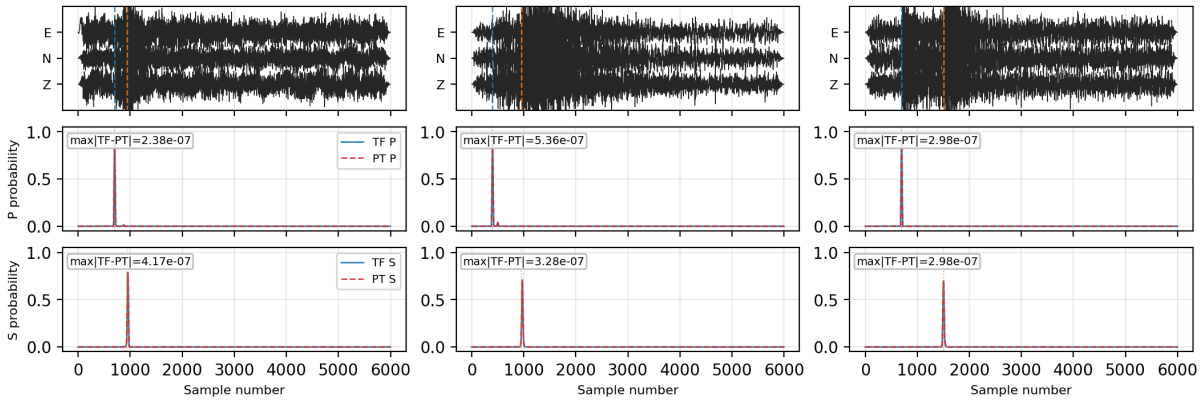

Figure 4: Qualitative waveform-level comparison on STEAD catalogue windows satisfying in-window P/S picks: three exemplary 6000-sample ZNE excerpts (normalized per channel row, Z/N/E labelled on the y-axis at vertically stacked traces) with dashed vertical markers at catalogue P (*blue*) and S (*orange*); overlaid beneath are temporal P- and then S-branch logits converted to calibrated probabilities comparing TensorFlow (solid blue) to PyTorch (dashed red), with boxed max $|\mathrm{TF} - \mathrm{PT}|$ annotations per branch.

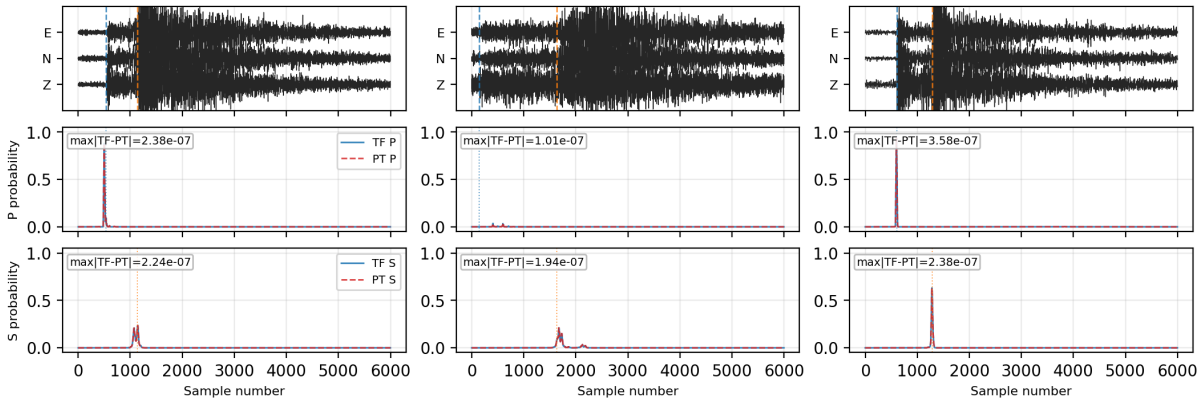

Figure 5: Same layout as Figure 4 but populated from TXED rather than STEAD.

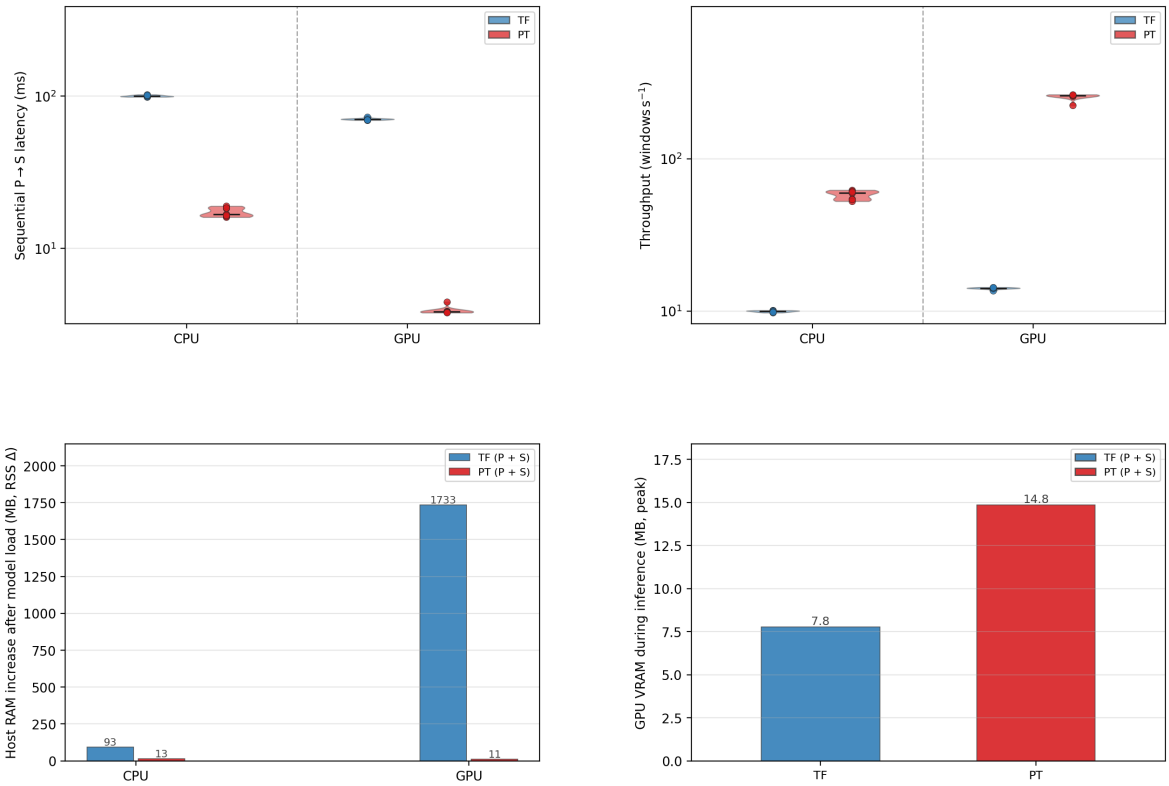

Figure 6: Controlled throughput and memory characterization on identical randomized 6000-sample ZNE tensors after warmup. *Panels A and B:* for each retained device facet (CPU and a single CUDA device in this plotted variant), empirical distributions (y-axis log scaled) comparing TensorFlow (*blue*) and PyTorch (*red*) on the summed sequential wall time ($t_{\mathrm{P}} + t_{\mathrm{S}}$) to produce both branch outputs (A, milliseconds) and the per-window reciprocal rate (B, windows s^{-1}). Strip markers coincide with categorical positions; dashed vertical separators distinguish CPU versus GPU facets. *Panel C:* additional host RSS () accrued when TF then PT checkpoints are staged in process memory. *Panel D:* peak accelerator memory during timed inference comparing TF versus PT on the GPU profile (CPU facets carry no analogous bars). Legends in the upper row denote backend only; legends in the lower row unify "TF (P + S)" and "PT (P + S)."

5.2 Practical considerations

Several practical issues are worth anticipating when reproducing or extending this work. Version control is critical: the specific versions of TensorFlow, Keras, CUDA, and GPU drivers should be pinned and documented alongside every released checkpoint pair, because even minor version changes can alter multi-head attention naming conventions or break strict weight loading. Environment consistency is equally important. If a notebook runs in a different conda environment than the validation scripts, TF may report no available GPUs or load a different library build, which can easily be mistaken for a modeling error.

As frameworks continue to evolve, not every layer conversion will follow a clean one-to-one mapping. Updates to TF may change attention bias layouts or multi-head attention naming, as seen in Keras 3, requiring corresponding adjustments to the loader. While EQCCT is representative of transformer-based pickers with convolutional front ends, models with different output heads or normalization schemes will need their own transformation tables and validation tests. The key lesson that can be learned from this work is how to build an automated layer-by-layer validation early in the weight transfer process so that failures that occur from specific operation mismatches can be quickly identified rather than dealing with vague discrepancies in end-to-end outputs.

5.3 When to convert versus retrain

Conversion is generally the most appropriate when the goal is to preserve an existing, field-tested checkpoint and make it accessible in a different ecosystem. It becomes less favorable to do so when the model requires structural changes such as new output heads, different normalization schemes, or modified input sampling, or when the training pipeline is already being revisited for other reasons. In those cases, retraining directly in the target framework is likely simpler than maintaining conversion code that must track two evolving implementations in parallel.

5.4 Generalizability

The workflow described here applies most directly to encoder-style phase pickers with static Keras graphs and weights stored in HDF5 checkpoints. EQCCT is a useful test case because it combines several sources of complexity: a dual-branch architecture, different layer depths between branches, and active use in production systems. Together these properties stress-test the conversion pipeline in ways that simpler models would not. The underlying principles, establishing canonical module definitions, applying systematic layout transformations, using name-based weight loading in the reference framework, and validating progressively from individual tensors to full waveforms, transfer readily to other institutional models even when specific layer configurations differ.

5.5 Limitations

This study demonstrates numerical agreement between two specific implementations of the same architecture under fixed preprocessing and windowing conditions. It does not establish that conversions will be equally straightforward for models with dynamic computational graphs, TF-only operators, quantized or heavily fused deployments, or architectures that rely on framework-specific behavior. The GPU results further illustrate that device kernels can introduce differences that are larger than those seen on CPU, even though they remain bounded. In contexts where bit-level reproducibility is required, the appropriate response is to constrain execution to CPU or to report device-specific tolerances explicitly, rather than assuming that any GPU stack will reproduce the original training environment exactly.

6 Data and code availability

Seismic waveform data was derived from the TXED and STEAD datasets that were made pubicaly accessible via SeisBench's API. Computing resources was provided by TexNet. All experiments were conducted on dual AMD Ryzen Threadripper PRO 7985WX CPUs with 512 GB RAM and two NVIDIA RTX 6000 Ada Generation GPUs, using the CPU affinity controls documented in the RAPID framework. EQCCT training artifacts and architecture definitions are distributed through the open EQCCT project; the Keras checkpoints we convert are from the same lineage as those used in operational studies ([29]). The TF-to-PT conversion code, validation scripts, and demonstration notebooks are available in the `eqcct2pt` Github repository. Reference artifacts include Keras HDF5 checkpoints for both P and S branches, along with optional PyTorch checkpoints generated by the validation utilities.

7 Conclusion

Institutional groups should not have to choose between years of investment in a TF-based workflow and participation in a PT-centered ecosystem. The workflow we present offers a reproducible path for converting TF-based models like EQCCT into a PT format that is compatible with SeisBench, with validation that spans from individual tensor layouts to full waveform comparisons. We hope it serves as a useful starting point for other groups facing the same challenge, and that it encourages similar documentation for other widely used phase pickers so that interoperability becomes a shared standard rather than an occasional effort.

Acknowledgments

The authors would like to thank the Texas Seismological Network and Seismology Research (TexNet) group and the State of Texas that provided financial support for this publication under the University of Texas at Austin award #201503664.

References

[1] Abadi, M., A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mane, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viegas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng (2015). TensorFlow: Large-scale machine learning on heterogeneous systems. Software available from tensorflow.org.

[2] Ali, M. (2023). Distributed processing using ray framework in python.

[3] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference, AFIPS '67 (Spring)*, New York, NY, USA, pp. 483–485. Association for Computing Machinery.

[4] Bates, D. and D. Watts (2008). *Nonlinear Regression Analysis and Its Applications*, pp. 32–66.

[5] Chen, Y., O. M. Saad, A. Savvaidis, F. Zhang, Y. Chen, D. Huang, H. Li, and F. Aziz Zanjani (2024). Deep learning for p-wave first-motion polarity determination and its application in focal mechanism inversion. *IEEE Transactions on Geoscience and Remote Sensing* **62**, 1–11.

[6] Chen, Y., Savvaidis, A., Siervo, D., Huang, D., & Saad, O. M. (2024). Near real-time earthquake monitoring in Texas using the highly precise deep learning phase picker. *Earth and Space Science*, **11**, e2024EA003890. `https://doi.org/10.1029/2024EA003890`

[7] Choi, S. B. (2025). Design and experimental research of on-device style transfer models based on PyTorch to ONNX. *Scientific Reports*, 15, Article number: 9345.

[8] Czyzewski, M. A. (2021). Transfer Learning Between Different Architectures Via Weights Injection. arXiv preprint arXiv:2101.02757.

[9] Feng, T., S. Mohanna, and L. Meng (2022). Edgephase: A deep learning model for multi-station seismic phase picking. *Geochemistry, Geophysics, Geosystems* **23**(11), e2022GC010453.

[10] Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *The Annals of Statistics* **29**(5), 1189–1232.

[11] Gustafson, J. L. (1988). Reevaluating amdahl's law. *Commun. ACM* **31**(5), 532–533.

[12] Herlihy, M. and N. Shavit (2012). *The Art of Multiprocessor Programming, Revised Reprint*. Morgan Kaufmann.

[13] Johnston, J. and J. DiNardo (1997). *Econometric Methods* (4th ed.). McGraw-Hill.

[14] Li, J., L. Ma, and Y. Lin (2023). An empirical study of challenges in converting deep learning models.

[15] Lim, C. S. Y., S. Lapins, M. Segou, and M. J. Werner (2025). Deep learning phase pickers: how well can existing models detect hydraulic-fracturing induced microseismicity from a borehole array? *Geophysical Journal International* **240**(1), 535–549. `https://doi.org/10.1093/gji/ggae386`

[16] McCool, M., J. Reinders, and A. Robison (2012). *Structured Parallel Programming: Patterns for Efficient Computation*. Elsevier Science.

[17] Moritz, P., R. Nishihara, S. Wang, A. Tumanov, R. Liaw, X. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, and I. Stoica (2018). Ray: A Distributed Framework for Emerging AI Applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, Carlsbad, CA, pp. 561–577.

[18] Mousavi, S., W. Ellsworth, Z. Weiqiang, L. Chuang, and G. Beroza (2020). Earthquake transformer—an attentive deep-learning model for simultaneous earthquake detection and phase picking. *Nature Communications* **11**, 3952.

[19] Mousavi, S. M. and G. C. Beroza (2022). Deep-learning seismology. *Science* **377**(6607), eabm4470.

[20] Muñoz, C., V. Salles, C. Skevofilax, and A. Savvaidis (2025). SCMLPick: A SeisComP Module Implementing Machine Learning Phase Picking for Real-Time Seismic Monitoring. [Manuscript submitted for publication]. Texas Seismological Network, University of Texas at Austin.

[21] Münchmeyer, J., J. Woollam, A. Rietbrock, F. Tilmann, D. Lange, T. Bornstein, T. Diehl, C. Giunchi, F. Haslinger, D. Jozinović, A. Michelini, J. Saul, and H. Soto (2022). Which picker fits my data? a quantitative evaluation of deep learning based seismic pickers. *Journal of Geophysical Research: Solid Earth* **127**.

[22] ONNX Community (2021). ONNX: Open Neural Network Exchange. Available at `https://onnx.ai/`

[23] Parida, S. K., and other authors (2025). How Do Model Export Formats Impact the Development of ML-Enabled Systems? A Case Study on Model Integration. arXiv preprint arXiv:2502.00429.

[24] Paszke, A., S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems* 32, 8024–8035.

[25] Ray.io (n.d.). What is ray core? `https://docs.ray.io/en/latest/ray-core/walkthrough.html`

[26] Saad, O. M. and Y. Chen (2022). Capsphase: Capsule neural network for seismic phase classification and picking. *IEEE Transactions on Geoscience and Remote Sensing* **60**, 1–11.

[27] Saad, O. M., Y. Chen, A. Savvaidis, W. Chen, F. Zhang, and Y. Chen (2022). Unsupervised deep learning for single-channel earthquake data denoising and its applications in event detection and fully automatic location. *IEEE Transactions on Geoscience and Remote Sensing* **60**, 1–10.

[28] Saad, O. M., Y. Chen, A. Savvaidis, S. Fomel, and Y. Chen (2022). Real-time earthquake detection and magnitude estimation using vision transformer. *Journal of Geophysical Research: Solid Earth* **127**(5), e2021JB023657.

[29] Saad, O. M., Y. Chen, D. Siervo, F. Zhang, A. Savvaidis, G.-c. D. Huang, N. Igonin, S. Fomel, and Y. Chen (2023). Eqcct: A production-ready earthquake detection and phase-picking method using the compact convolutional transformer. *IEEE Transactions on Geoscience and Remote Sensing* **61**, 1–15.

[30] Saad, O. M., G. Huang, Y. Chen, A. Savvaidis, S. Fomel, N. Pham, and Y. Chen (2021). Scalodeep: A highly generalized deep learning framework for real-time earthquake detection. *Journal of Geophysical Research: Solid Earth* **126**(4), e2020JB021473.

[31] Savvaidis, A., B. Young, G. D. Huang, and A. Lomax (2019). Texnet: A statewide seismological network in texas. *Seismological Research Letters* **90**(4), 1702–1715.

[32] SecondTalent (n.d.). PyTorch vs. TensorFlow: usage, popularity, and performance. `https://www.secondtalent.com/resources/pytorch-vs-tensorflow-usage-popularity-and-performance/`

[33] Sheen, D.-H., Y.-J. Seong, S. Jung, and J.-h. Park (2023). Real-time application of deep learning seismic phase picker to earthquake monitoring. Poster S31G-0424, *AGU Fall Meeting*, San Francisco, CA.

[34] Skevofilax, C., Muñoz, C., V. Salles, and A. Savvaidis (2026). RAPID: A Generalized High-Performance Parallelization Framework for Real-Time Deep Learning Seismic Phase Picking. [Unpublished manuscript]. Texas Seismological Network, University of Texas at Austin.

[35] Tan, Y. J., F. Waldhauser, W. Ellsworth, M. Zhang, Z. Weiqiang, M. Michele, L. Chiaraluce, G. Beroza, and M. Segou (2021). Machine-learning-based high-resolution earthquake catalog reveals how complex fault structures were activated during the 2016–2017 central italy sequence. *The Seismic Record* **1**, 11–19.

[36] Verma, G., S. Raskar, M. Emani, and B. Chapman (2024). Cross-Feature Transfer Learning for Efficient Tensor Program Generation. *Applied Sciences*, 14(2), 513.

[37] Wang, T., D. Trugman, and Y. Lin (2021). Seismogen: Seismic waveform synthesis using gan with application to seismic data augmentation. *Journal of Geophysical Research: Solid Earth* **126**(4), e2020JB020077.

[38] Woollam, J., J. Münchmeyer, F. Tilmann, A. Rietbrock, D. Lange, T. Bornstein, T. Diehl, C. Giunchi, F. Haslinger, and D. Jozinović (2022). SeisBench—A Toolbox for Machine Learning in Seismology. *Seismological Research Letters* **93**(3), 1695–1709.

[39] Xiao, Z., J. Wang, C. Liu, J. Li, L. Zhao, and Z. Yao (2021). Siamese earthquake transformer: A pair-input deep-learning model for earthquake detection and phase picking on a seismic arrival-time picking method. *Journal of Geophysical Research: Solid Earth* **126**(5), e2020JB021444.

[40] Yeck, W. L., J. M. Patton, Z. E. Ross, G. P. Hayes, M. R. Guy, N. B. Ambruz, D. R. Shelly, H. M. Benz, and P. S. Earle (2020). Leveraging Deep Learning in Global 24/7 Real-Time Earthquake Monitoring at the National Earthquake Information Center, *Seismol. Res. Lett.* **92**, 469–480, doi: 10.1785/0220200178.

[41] Yu, Z., Y. Jiang, X. Jing, and H. Zheng (2024). Study on geomagnetic observations associated with three major earthquakes in southwest china through a novel deep learning framework. *IEEE Transactions on Geoscience and Remote Sensing* **62**, 1–18.

[42] Zhang, Y., Z. Jia, et al. (2025). Bridging Operator Semantic Inconsistencies: A Source-Level Cross-Framework Model Conversion Approach

[43] Zhu, W. and G. C. Beroza (2018). Phasenet: a deep-neural-network-based seismic arrival-time picking method. *Geophysical Journal International* **216**(1), 261–273.